



NUMPY

~ Arijit Das



```
import random
print(random.choice([
    "You can do it!",
    "Keep going!",
    "Believe in yourself!",
    "Stay positive!",
    "One step at a time."
]))
```

#Quote #Programming #Selfcare

Definition of Numpy:

Numpy is an open source python library used for scientific computing. It provides efficient array objects, mathematical functions, and tools for working with large n-dimensional arrays.

Key features:

- Vectorised operation → element wise operation without using loop.
- Broadcasting → operations on arrays on different shapes.
- Mathematical function → linear algebra, statistics, trigonometry.
- Integration → Works well with other libraries like Pandas, SciPy, Matplotlib and Seikid-learn.

Feature	<u>List</u>	<u>Numpy-Array</u>
Type	Built-in data structure	(i) provided by numpy library.
(i) Data types	Can store multiple data types.	(ii) Stores homogenous data.
(ii) Performance	More memory	(iv) Less memory.
(iv) Functionality	Basic math function	(iv) Advanced math function.
(v) Use-case	General purpose	(v) Numerical data computation, ML, data analysis.

Installation:

`pip install numpy` in your command prompt

Use case:

- It is used for working with arrays.
- It is short for 'Numerical python'.
- Smaller memory consumption.

After installation:

- `import numpy as np`

① Creating arrays

1D - Array

```
arr1 = np.array([10, 20, 30, 40])  
print(arr1)
```

2D - Array

```
arr1 = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
print(arr1)
```

3D - Array

```
arr1 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  
print(arr1)
```

② Creation of arrays-2 with Syntax

(i) # zeros

```
arr1 = np.zeros((3,3))  
print(arr1)
```

(ii) # ones

```
arr2 = np.ones((3,3))  
print(arr2)
```

(iii) # full

```
arr3 = np.full((2,3), 7)  
print(arr3)
```

(iv) # identity

```
arr4 = np.eye(3)  
print(arr4)
```

(v) # arange

```
arr5 = np.arange(1,11,1)  
print(arr5)
```

(vi) # linspace

```
arr6 = np.linspace(0,1,5)  
print(arr6)
```


Array attributes

i) arr.shape
ii) arr.dtype
iii) arr.ndim
iv) arr.size

Array slicing & indexing

i) 1-D
arr = np.array([10, 20, 30, 40])

Indexing

print(arr[0])
print(arr[3])
print(arr[2])

Slicing

print(arr[0:1])
print(arr[0:2])
print(arr[0:3])
print(arr[0:4])

2-D

arr2 = np.array([[1, 2, 3], [4, 5, 6]])

Indexing

print(arr2[row idx, col idx])
print(arr2[0, 2]) # 3
print(arr2[1, 1]) # 5

Slicing

(arr2[row idx: col idx])
print(arr2[0:1, 2:]) # [[3]]
print(arr2[1:, 0:1]) # [[4]]

[[1 2 3]]	→ Row 0
[4 5 6]]	→ Row 1
↓ ↓ ↓	
col 0 col 1 col 2	

⑤ Mathematical Operation

```
print(np.add(arr1, arr2))  
print(np.subtract(arr1, arr2))  
print(np.divide(arr1, arr2))  
print(np.multiply(arr1, arr2))  
print(np.power(arr1, 2))
```

→ Math functions

```
print(np.sum(arr1))  
print(np.median(arr1))  
print(np.mode(arr1))  
print(np.min(arr1))  
print(np.max(arr1))  
print(np.mean(arr1))  
print(np.std(arr1))
```

→ Aggregate functions

```
print(np.round(arr1, 2)) # 1.5927 → 1.59  
print(np.ceil(arr1)) # 1.2 → 2  
print(np.floor(arr1)) # 1.5 → 1  
print(np.trunc(arr1)) # 1.9 → 1
```

→ Rounding function

```
print(np.cumsum(arr1))  
print(np.cumprod(arr1))
```

→ Cumulative operation

np.abs([-1, -2, -3]) # [1 2 3]
np.sign([-5, 0, 4]) # [-1 0 1]

→ Absolute & sign

Reshaping & Manipulation

arr.reshape(2, 3) # change shape

arr.flatten() # 1D array

arr.concatenate([arr1, arr2]) # merge two array

arr.vstack((arr1, arr2)) # stack vertically

arr.hstack(arr1, arr2) # stack horizontally

Insert, Delete, Append

print(np.insert(arr1, index, [elements], axis))

print(np.delete(arr1, index))

print(np.append(arr1, values))

Searching & Sorting

np.where(arr > 5)

np.sort(arr)

np.argsort(arr)

Trigonometric function

np.sin(arr)

np.cos(arr)

np.tan(arr)

np.exp(arr)

```
import numpy as np
#zeros
arr = np.zeros((3,3))
print(arr)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

```
#ones

arr2 = np.ones((3,4))
print(arr2)
```

```
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
```

```
#full

arr3 = np.full((3,3),5)
print(arr3)
```

```
[[5 5 5]
 [5 5 5]
 [5 5 5]]
```

#identity matrix

```
arr4 = np.eye(3,3)
print(arr4)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

#arange

```
arr5 = np.arange(1, 11, 1)
print(arr5)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

#linspace

```
arr6 = np.linspace(0, 1, 5)
print(arr6)
```

```
[0.    0.25 0.5   0.75 1.   ]
```

```
import numpy as np
#1D-Array
arr = np.array([1,2,3,4,5,6])
print(arr)
print(arr.dtype)
print(arr.ndim)
print(arr.size)
print(arr.shape)
```

```
[1 2 3 4 5 6]
int64
1
6
(6,)
[1 2 3 4 5 6]
float64
1
4
(4,)
```

```
#2D-Array
arr2 = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(arr2)
print(arr2.dtype) #data type
print(arr2.ndim) #n-dimension
print(arr2.size) #number of elements present in the array
print(arr2.shape) #shape of the array
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
int64
2
9
(3, 3)
```



```
#conversion
```

```
#1D-Array
```

```
print(arr.astype(float)) #converts integer to float
```

```
print(arr.astype(str)) #converts integer to string
```

```
[1. 2. 3. 4. 5. 6.]
```

```
['1' '2' '3' '4' '5' '6']
```

```
#2D-Array
```

```
print(arr2.astype(float)) #converts integer to float
```

```
print(arr2.astype(str)) #converts integer to string
```

```
[[1. 2. 3.]
```

```
 [4. 5. 6.]
```

```
 [7. 8. 9.]]
```

```
 [['1' '2' '3']
```

```
  ['4' '5' '6']
```

```
  ['7' '8' '9']]
```

```
#slicing of arrays
#1D-Array
import numpy as np

arr = np.array([1,2,3,4,5])

print(arr[0:])
print(arr[0:1])
print(arr[0:2])
print(arr[0:3])
print(arr[0:4])
print("\n")
#accessing specific elements from array
print(arr[0])
print(arr[1])
print(arr[2])
print(arr[3])
print(arr[4])
```

[1 2 3 4 5]

[1]

[1 2]

[1 2 3]

[1 2 3 4]

1

2

3

4

5


```
#2D-Array
arr1 = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(arr1)
#accessing rows only
#row , column
print(arr1[0:1,])
print(arr1[0:2])
print(arr1[0:3])
#accessing columns only
print(arr1[0:3,2:])
print(arr1[0:3,1:2])
print(arr1[0:3,0:1])
#accessing specific elements
print(arr1[0:1,0])
print(arr1[0:1,1])
print(arr1[0:1,2])
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[1 2 3]]
[[1 2 3]
 [4 5 6]]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[3]
 [6]
 [9]]
[[2]
 [5]
 [8]]
[[1]
 [4]
 [7]]
[1]
[2]
```

```

#arithmetic functions
#1D-Array
import numpy as np
arr1 = np.array([1,2,3,4,5])
arr2 = np.array([6,7,8,9,10])
print(arr1+arr2)
print(arr1-arr2)
print(arr1*arr2)
print(arr1/arr2)
print("\n")
#using functions
print(np.add(arr1,arr2))
print(np.subtract(arr1,arr2))
print(np.divide(arr1,arr2))
print(np.multiply(arr1,arr2))
print(np.power(arr1,arr2)) #arr1 to the power arr2
#2D-Array
arr4 = np.array([[1,2,3],
                 [4,5,6],
                 [7,8,9]])

arr5 = np.array([[10,11,12],
                 [13,14,15],
                 [16,17,18]])

print(arr1+arr2)
print(arr1-arr2)
print(arr1*arr2)
print(arr1/arr2)
print("\n")
#using functions
print(np.add(arr4,arr5))
print(np.subtract(arr1,arr2))
print(np.divide(arr1,arr2))
print(np.multiply(arr1,arr2))
print(np.power(arr1,arr2)) #arr1 to the power arr2

```

```
[ 7  9 11 13 15]
[-5 -5 -5 -5 -5]
[ 6 14 24 36 50]
[0.16666667 0.28571429 0.375      0.44444444 0.5      ]
```

```
[ 7  9 11 13 15]
[-5 -5 -5 -5 -5]
[0.16666667 0.28571429 0.375      0.44444444 0.5      ]
[ 6 14 24 36 50]
[      1      128      6561      262144 9765625]
[ 7  9 11 13 15]
[-5 -5 -5 -5 -5]
[ 6 14 24 36 50]
[0.16666667 0.28571429 0.375      0.44444444 0.5      ]
```

```
[[11 13 15]
 [17 19 21]
 [23 25 27]]
[-5 -5 -5 -5 -5]
[0.16666667 0.28571429 0.375      0.44444444 0.5      ]
[ 6 14 24 36 50]
[      1      128      6561      262144 9765625]
```

#aggregate functions

#1D-Array

```
arr6 = np.array([1,2,3,4,5])  
arr7 = np.array([6,7,8,9,10])
```

#sum

```
print(np.sum(arr6))  
print(np.sum(arr7))  
print(np.sum([arr6,arr7]))
```

#min & max

```
print(np.min(arr6))  
print(np.max(arr7))  
print(np.max([arr6,arr7])) #when it find the max element it first concat the elements horizontally.
```

#mean median

```
print(np.mean(arr6))  
print(np.mean(arr7))  
print(np.median(arr6))  
print(np.median(arr7))
```

#median

#if the array is odd then it takes 1 middle element and print it.

#if the array is even it takes two middle two element and add those two then divide by 2.

#std = it tells how much differ the elements from average.

```
print(np.std(arr6))  
print(np.std(arr7))
```

#argmax = prints the index of max element of the array.

#argmin = prints the index of min element of the array.

```
print(np.argmax(arr6)) #max element = 5[4]  
print(np.argmin(arr7)) #min element = 6[0]
```

```
#2D-Array
arr8 = np.array([[1,2,3],[4,5,6],[7,8,9]])

#sum
print(np.sum(arr8, axis=1))
print(np.sum(arr8, axis=0))

#min & max
print(np.min(arr8))
print(np.max(arr8))
print(np.min(arr8, axis=1))
print(np.max(arr8, axis=0))

#mean median
print(np.mean(arr8))
print(np.median(arr8))
print(np.mean(arr8, axis=1))
print(np.median(arr8, axis=0))

#std = it tells how much differ the elements from average.
print(np.std(arr8))
print(np.std(arr8))

#argmax = prints the index of max element of the array.
#argmin = prints the index of min element of the array.
print(np.argmax(arr8)) #max element = 9[8]
print(np.argmin(arr8)) #min element = 1[0]
```

```
15
40
55
1
10
10
3.0
```

15
40
55
1
10
10
3.0
8.0
3.0
8.0
1.4142135623730951
1.4142135623730951
4
0
[6 15 24]
[12 15 18]
1
9
[1 4 7]
[7 8 9]
5.0
5.0
[2. 5. 8.]
[4. 5. 6.]
2.581988897471611
2.581988897471611
8
0

#rounding functions

#ROUND = it round off the decimal places.

#1D-Array

```
arr9 = np.array([1.2345, 45.6789, 12.5678])  
print(np.round(arr9, 2))
```

#2D-Array

```
arr10 = np.array([[1.456, 2.789], [3.123, 4.987]])  
print(np.round(arr10, 1))
```

#CEIL = it rounds up to the nearest integer.

#1D-Array

```
print(np.ceil(arr9))
```

#2D-Array

```
print(np.ceil(arr10))
```

#FLOOR = it rounds downs to the nearest integer.

#1D-Array

```
print(np.floor(arr9))
```

#2D-Array

```
print(np.floor(arr10))
```

#TRUNCATE = It just cut off the decimal.

#1D-Array

```
print(np.trunc(arr9))
```

#2D-Array

```
print(np.trunc(arr10))
```

[1.23 45.68 12.57]

[[1.5 2.8]

[3.1 5.]]

[2. 46. 13.]

[[2. 3.]

[4. 5.]]

[1. 45. 12.]

[[1. 2.]

[3. 4.]]

[1. 45. 12.]

[[1. 2.]

[3. 4.]]

```

#concat
#1D-Array
import numpy as np

arr1 = np.array([1,2,3,4,5])
arr2 = np.array([6,7,8,9,10])

print(np.concatenate([arr1,arr2]))

#vstack() and hstack()
print(np.vstack(arr1)) #vertically
print(np.hstack(arr2)) #horizontally

```

```

[ 1  2  3  4  5  6  7  8  9 10]
[[1]
 [2]
 [3]
 [4]
 [5]]
[ 6  7  8  9 10]

```

```

#2D-Array

arr3 = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(arr3)

print(np.vstack(arr3))
print(np.hstack(arr3))

```

```

[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[1 2 3 4 5 6 7 8 9]

```

#concat-2D-Array

```
arr4 = np.array([[1,2,3],[4,5,6],[7,8,9]])
arr5 = np.array([[10,11,12],[13,14,15],[16,17,18]])
print(np.concatenate([arr4,arr5])) #by default vertically
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]
 [13 14 15]
 [16 17 18]]
```

#understanding axis of 2D array

#2D-Array has two dimensions and two axis.1 and 0.

#0 represents vertically means column wise.

#1 represents horizontally means row wise.

#axis = 1 means when it done its operation row wise then it change its direction to the next row(vertically changing its direction)

#axis = 0 means when it done its operation column wise then it change its direction to the next column(horizontally changing its direction)

```
arr6 = np.array([[1,2,3],[4,5,6],[7,8,9]])
arr7 = np.array([[10,11,12],[13,14,15],[16,17,18]])
```

```
print(np.concatenate([arr6,arr7],axis=0))
print(np.concatenate([arr6,arr7],axis=1))
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]
 [13 14 15]
 [16 17 18]]
[[ 1  2  3 10 11 12]
 [ 4  5  6 13 14 15]
 [ 7  8  9 16 17 18]]
```



```
arr7 = np.array([[10,11,12],[13,14,15],[16,17,18]])
```

```
print(np.concatenate([arr6,arr7],axis=0))
```

```
print(np.concatenate([arr6,arr7],axis=1))
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]
 [13 14 15]
 [16 17 18]]
[[ 1  2  3 10 11 12]
 [ 4  5  6 13 14 15]
 [ 7  8  9 16 17 18]]
```

#using vstack and hstack.

```
print(np.vstack([arr6,arr7]))
```

```
print(np.hstack([arr6,arr7]))
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]
 [13 14 15]
 [16 17 18]]
[[ 1  2  3 10 11 12]
 [ 4  5  6 13 14 15]
 [ 7  8  9 16 17 18]]
```

```
import numpy as np
```

```
#1D-Array
```

```
arr1 = np.array([1,2,3,4,5])
```

```
print(np.append(arr1, 6))
```

```
[1 2 3 4 5 6]
```

```
print(np.append(arr1, [6,7,8,9]))
```

```
[1 2 3 4 5 6 7 8 9]
```

```
#2D-Array
```

```
arr3 = np.array([[1,2,3],  
                 [4,5,6],  
                 [7,8,9]])
```

```
print(np.append(arr3, [10,11,12]))
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

```
#np.delete(array_name, index, [element], axis)
```

```
print(np.insert(arr3 , 2 , [7,8,9] , axis=0))
```

```
[[1 2 3]  
 [4 5 6]  
 [7 8 9]  
 [7 8 9]]
```

```
arr2 = np.array([[1,2,3],
                 [4,5,6],
                 [7,8,9],
                 [10,11,12],
                 [13,14,15]])
print(np.insert(arr2,4,[16,17,18],axis=0))
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]
 [16 17 18]
 [13 14 15]]
```

```
print(arr2)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]
 [13 14 15]]
```

#if you do not declare axis , it will be convert into a 1D-Array then delete the element by index.
#syntax = np.delete(array_name, index)

```
print(np.delete(arr2, 1))
```

```
[ 1  3  4  5  6  7  8  9 10 11 12 13 14 15]
```

#if we declare axis,
#np.delete(array_name, index, axis)
print(np.delete(arr2, 1, axis=1)) #column

```
[[ 1  3]
 [ 4  6]
 [ 7  9]
 [10 12]
 [13 15]]
```

```
print(np.delete(arr2, 1, axis=0)) #row
```

```
[[ 1  2  3]
 [ 7  8  9]
 [10 11 12]
 [13 14 15]]
```

```
print(np.delete(arr2,[0,1] , axis=1))
```

```
[[ 3]
 [ 6]
 [ 9]
 [12]
 [15]]
```

```
print(np.delete(arr2, [0,1], axis=0))
```

```
[[ 7  8  9]
 [10 11 12]
 [13 14 15]]
```



```
import numpy as np
#1D-Array
#sort
arr1 = np.array([10,9,8,7,6,5,4,3,2,1])
```

```
print(np.sort(arr1)) #ascending order
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

```
#2D-Array
#sort
arr2= np.array([[10,9,8,7,6,5,4,3,2,1],
                [20,19,18,17,16,15,14,13,12,11]])
```

```
print(np.sort(arr2))
```

```
[[ 1  2  3  4  5  6  7  8  9 10]
 [11 12 13 14 15 16 17 18 19 20]]
```

```
arr3= np.array([[10,9,8],
                [7,6,5],
                [4,3,2],
                [1,20,19],
                [18,17,16],
                [15,14,13]])
```

```
print(np.sort(arr3))
```

```
[[ 8  9 10]
 [ 5  6  7]
 [ 2  3  4]
 [ 1 19 20]
 [16 17 18]
 [13 14 15]]
```

```
#search
```

```
arr4 = np.array([[10,9,8],  
                 [7,6,5],  
                 [5,3,2]])
```

```
print(np.where(arr4==7))
```

```
print(np.where(arr4==5))
```

```
print(np.where(arr4==2))
```

```
#print by the index
```

```
#1st = row , 2nd= column
```

```
(array([1]), array([0]))
```

```
(array([1, 2]), array([2, 0]))
```

```
(array([2]), array([2]))
```

```
arr5 = np.array([10,9,8,7,6,5,4,3,2,1])
```

```
print(np.where(arr5==6))
```

```
print(np.where(arr5==1))
```

```
print(np.where(arr5==3))
```

```
print(np.where(arr5==10))
```

```
(array([4]),)
```

```
(array([9]),)
```

```
(array([7]),)
```

```
(array([0]),)
```

```
c = np.where(arr5%2==0)
```

```
print(c) #filter out
```

```
(array([0, 2, 4, 6, 8]),)
```

```
arr5 = np.array([10,9,8,7,6,5,4,3,2,1])
```

```
sort = np.sort(arr5)
```

```
print(sort)
```

```
print(np.searchsorted(sort, 3))
```

#print the index after sorted.

#it only works when the array is sorted.

```
[ 1  2  3  4  5  6  7  8  9 10]
```

```
2
```

#filter

```
arr5 = np.array([10,9,8,7,6,5,4,3,2,1])
```

```
even_arr5 = arr5%2==0
```

```
odd_arr5 = arr5%2!=0
```

```
print(np.sort(arr5[even_arr5]))
```

```
print(np.sort(arr5[odd_arr5]))
```

```
[ 2  4  6  8 10]
```

```
[1 3 5 7 9]
```

```
#Daily financial progress.
```

```
#cumcum
```

```
import numpy as np
```

```
target_earning = 7000
```

```
print("target_earning : ",target_earning)
```

```
income_perday = np.array([500, 450, 780, 1200, 600, 700, 2300])
```

```
print("Per day income : ",income_perday)
```

```
cumsum = np.cumsum(income_perday)
```

```
print("Total earning each day : ",cumsum)
```

```
total = np.sum(income_perday)
```

```
print("Total income : ",total)
```

```
remain_earning = target_earning-total
```

```
print("Remaining earning : ",remain_earning)
```

```
target_earning : 7000
```

```
Per day income : [ 500  450  780 1200  600  700 2300]
```

```
Total earning each day : [ 500  950 1730 2930 3530 4230 6530]
```

```
Total income : 6530
```

```
Remaining earning : 470
```

#Tracking Study Hours Over a Week

```
total_hours = 168
print("Total hours in 1 week : ",total_hours)

studied_each_week = np.array([6, 12, 13, 11, 8, 6, 9])
print("I studied each week : ",studied_each_week)

total_hours_per_week = np.cumsum(studied_each_week)

total_hours_i_studied = np.sum(studied_each_week)
print("Total hours i studied : ",total_hours_i_studied)

remaining_time = total_hours - total_hours_i_studied
print("Remaining time : ",remaining_time)
```

```
Total hours in 1 week : 168
I studied each week : [ 6 12 13 11  8  6  9]
Total hours i studied : 65
Remaining time : 103
```

#cumprod

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])
print(np.cumprod(arr))
```

```
[      1      2      6     24    120    720   5040  40320 362880
 3628800]
```



```
#cumprod
```

```
arr = np.array([1,2,3,4,5,6,7,8,9,10])  
print(np.cumprod(arr))
```

```
[      1      2      6     24    120    720   5040  40320 362880  
 3628800]
```

```
#You invest ₹1,000 in a fund that grows every year by different percentages.
```

```
import numpy as np
```

```
initial_money = 1000  
growth_factors = np.array([1.10, 1.20, 1.05, 1.15, 1.25])  
cumulative_growth = initial_money * np.cumprod(growth_factors)  
  
print(cumulative_growth)
```

```
[1100.    1320.    1386.    1593.9   1992.375]
```

*#You have 1000 random ages between 18 and 65.
#Find the percentage of people in the 18-25, 26-40, and 41-65 ranges.*

```
import numpy as np

arr = np.random.randint(18, 66, size=1000)

total = arr.size #1000

from_18_25 = np.sum((arr>=18) & (arr<=25))/total * 100
from_26_40 = np.sum((arr>=26) & (arr<=40))/total * 100
from_41_65 = np.sum((arr>=41) & (arr<=65))/total * 100

print(f"From 18 to 25 is : {from_18_25}")
print(f"From 26 to 40 is : {from_26_40}")
print(f"From 41 to 65 is : {from_41_65}")
```

```
From 18 to 25 is : 16.7
From 26 to 40 is : 31.7
From 41 to 65 is : 51.6
```

*#Create a monthly sales dataset for 3 products over 12 months.
#Find the month with the highest overall sales.*

```
arr = np.random.randint(1000, 2000, size=(3,12))  
print(arr)
```

```
print()
```

```
p1 = arr[0:1, :12]  
print("Product 1 sale of 12 months : ", p1)  
p2 = arr[1:2, :12]  
print("Product 2 sale of 12 months : ", p2)  
p3 = arr[2:, :12]  
print("Product 3 sale of 12 months : ", p3)
```

```
print()
```

#Highest overall sales.

```
print("Highest sale of product 1 : ", np.max(p1))  
print("Highest sale of product 2 : ", np.max(p2))  
print("Highest sale of product 3 : ", np.max(p3))
```

```
print()
```

```
[[1342 1982 1861 1850 1571 1459 1788 1890 1113 1006 1704 1586]
 [1482 1691 1338 1955 1253 1460 1115 1982 1411 1053 1665 1349]
 [1081 1607 1534 1594 1281 1633 1414 1257 1614 1493 1418 1924]]
```

```
Product 1 sale of 12 months :  [[1342 1982 1861 1850 1571 1459 1788 1890 1113 1006 1704 1586]]
Product 2 sale of 12 months :  [[1482 1691 1338 1955 1253 1460 1115 1982 1411 1053 1665 1349]]
Product 3 sale of 12 months :  [[1081 1607 1534 1594 1281 1633 1414 1257 1614 1493 1418 1924]]
```

```
Highest sale of product 1 : 1982
Highest sale of product 2 : 1982
Highest sale of product 3 : 1924
```

```
#Simulate stock prices for 10 days (np.random.randint(100, 200, size=10)).  
#Calculate the moving average.
```

```
arr = np.random.randint(100, 200, size=10)  
print(np.mean(arr))
```

164.8

```
#data = np.array([1, 2, np.nan, 4, np.nan, 6])  
#Replace NaN values with the mean of the non-NaN elements.
```

```
data = np.array([1, 2, np.nan, 4, np.nan, 6])  
print(data)
```

```
print()
```

```
data2 = np.nanmean(data) #3.25
```

```
print()
```

```
data[np.isnan(data)] = data2
```

```
print(data)
```

[1. 2. nan 4. nan 6.]

[1. 2. 3.25 4. 3.25 6.]


```
#data = np.array([1, -2, np.nan, 4, np.nan, -6])  
#Replace NaN values and negative values with the mean of the non-NaN elements.
```

```
data = np.array([1, -2, np.nan, 4, np.nan, -6])  
print("ID : ",data)
```

```
avg_val1 = np.nanmean(data)  
avg_val2 = abs(avg_val1)
```

```
print()
```

```
data[np.isnan(data)] = avg_val2  
data[data<0] = avg_val2
```

```
print(data)
```

```
ID : [ 1. -2. nan  4. nan -6.]
```

```
[1.  0.75 0.75 4.  0.75 0.75]
```

```
import numpy as np

#beginner level
#get the quaterly totals sales of each months.

sales = np.array([1000, 1500, 1200, 2000, 1800, 1700,
                  2200, 2100, 2300, 2500, 2400, 2600])

sales_quarters = sales.reshape(4, 3)

print()

print("January , February, March : ",sales_quarters[0:1, 0:3])
print("Epril , May, June : ",sales_quarters[1:2, 0:3])
print("July , August, September : ",sales_quarters[2:3 , 0:3])
print("october , November, December : ",sales_quarters[3:4, 0:3])

print()

quaterly_sales = np.sum(sales_quarters, axis=1)
print("Total sales per three months : ", quaterly_sales)
```

```
January , February, March :  [[1000 1500 1200]]
Epril , May, June :  [[2000 1800 1700]]
July , August, September :  [[2200 2100 2300]]
october , November, December :  [[2500 2400 2600]]

Total sales per three months :  [3700 5500 6600 7500]
```

```
random_array = np.random.randint(1, 100, size = 20)

print(random_array)

for i in random_array:

    if i>50:

        print(i)
```

```
[ 8 64 95 48 21 56 56 82 10 81 83 43 38 18  2 34 98 86 66 90]
64
95
56
56
82
81
83
98
86
66
90
```

#Apply a 10% discount to all products priced above 200.

```
prices = np.array([100, 200, 150, 300, 250])
```

```
for i in prices:
```

```
    if i>200:
```

```
        discount = (i/100)*10
```

```
        final_price = i-discount
```

```
        print("Discount : ",final_price)
```

Discount : 270.0

Discount : 225.0

#Find the top 3 highest salaries.

```
salaries = np.array([35000, 42000, 50000, 28000, 60000, 45000])
```

#first - sort the array in ascending order.

#then in descending order.

#then print the last three numbers.

```
asc_salary = np.sort(salaries) #in ascending order first.
```

```
desc_salary = asc_salary[::-1] #in descending order.
```

```
last_three_salary = desc_salary[0:3]
```

```
print(last_three_salary)
```

[60000 50000 45000]

```
#Simulate daily temperature data for 30 days (integers between 20 and 40).  
#Find:  
#Average weekly temperature.  
#Hottest day's temperature.
```

```
temp_data = np.random.randint(20, 40, size=30)
```

```
print(temp_data)
```

```
print()
```

```
reshape_days = temp_data.reshape(6,5)
```

```
print(reshape_days)
```

```
print()
```

```
total_temp_of_30_days = np.sum(reshape_days)
```

```
print("Total temperature of last 30 days : ",total_temp_of_30_days)
```

```
[22 34 38 24 27 28 23 37 33 38 34 22 20 31 23 30 27 35 30 37 39 35 34 27  
 31 30 22 37 33 32]
```

```
[[22 34 38 24 27]  
 [28 23 37 33 38]  
 [34 22 20 31 23]  
 [30 27 35 30 37]  
 [39 35 34 27 31]  
 [30 22 37 33 32]]
```

```
Total temperature of last 30 days : 913
```

#Find percentage growth for each quarter.

```
sales_2024 = np.array([250, 300, 400, 500])
```

```
sales_2025 = np.array([300, 350, 450, 550])
```

```
percentage_growth = ((sales_2025 - sales_2024)/sales_2024)*100
```

```
print(np.round(percentage_growth))
```

```
[20. 17. 12. 10.]
```



```
#Create a 5x5 matrix of integers 1-25. Extract:  
#First row  
#Last column  
#The diagonal elements
```

```
arange_arr = np.arange(1, 26)  
reshape_arr = arange_arr.reshape(5, 5)
```

```
print(reshape_arr)
```

```
print()
```

```
#First row  
print(reshape_arr[0, 0:5])
```

```
#Last column  
print(reshape_arr[0:5, 4])
```

```
#The diagonal elements  
print(np.diag(reshape_arr))
```

```
[[ 1  2  3  4  5]  
 [ 6  7  8  9 10]  
 [11 12 13 14 15]  
 [16 17 18 19 20]  
 [21 22 23 24 25]]
```

```
[1 2 3 4 5]  
[ 5 10 15 20 25]  
[ 1  7 13 19 25]
```

```
import numpy as np

matrix = np.arange(1, 26).reshape(5, 5)

print(matrix)

print()

#diagonal
diag = np.diag(matrix)
print("Diagonal:",diag)

print()

# Upper triangle
upper_triangle = np.triu(matrix)
print("Upper Triangle:\n", upper_triangle)

print()

# Lower triangle
lower_triangle = np.tril(matrix)
print("Lower Triangle:\n", lower_triangle)
```

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]
 [21 22 23 24 25]]
```

```
Diagonal: [ 1  7 13 19 25]
```

```
Upper Triangle:
```

```
[[ 1  2  3  4  5]
 [ 0  7  8  9 10]
 [ 0  0 13 14 15]
 [ 0  0  0 19 20]
 [ 0  0  0  0 25]]
```

```
Lower Triangle:
```

```
[[ 1  0  0  0  0]
 [ 6  7  0  0  0]
 [11 12 13  0  0]
 [16 17 18 19  0]
 [21 22 23 24 25]]
```